



Gestion des Tickets d'Incidents

Projet de Programmation - POO & Base de données

Enseignant : M. DIALLO
Classe : Licence 2 - Génie Logiciel (GL)
email : mmdialloisidk1@groupeisi.com
Date : 17 mai 2026

Travail à rendre par binôme ou trinôme

Informations générales

- **Objectif** : Développer une application console en Python (POO) avec une base de données PostgreSQL ou MySQL.
- **Contraintes** : 3 tables maximum, interface en ligne de commande, gestion des rôles.
- **Durée** : 4 semaines
- **Modalités** : Travail en binôme ou trinôme (groupes libres)
- **Rendu** : Dépôt GitHub + Présentation orale en classe

Contexte

La DSI (Direction des Systèmes d'Information) d'une grande entreprise sénégalaise souhaite mettre en place un système de gestion des incidents informatiques (Help Desk). Actuellement, les utilisateurs signalent leurs problèmes par email ou par téléphone, ce qui entraîne une perte de traçabilité, des délais de traitement longs et une absence de statistiques.

Vous êtes chargé de développer une application qui permettra :

- Aux utilisateurs de signaler des incidents
- Aux techniciens de prendre en charge et résoudre les incidents
- À l'administrateur de gérer les utilisateurs et de superviser l'ensemble

Structure de la base de données (3 tables)

Table utilisateur

- **id** : clé primaire, auto-incrémentée
- **login** : unique, obligatoire (ex : jdupont, mmbaye)
- **password** : obligatoire (stocké en clair pour ce projet)
- **nom** : obligatoire
- **prenom** : obligatoire
- **email** : obligatoire, format email valide
- **role** : UTILISATEUR, TECHNICIEN ou ADMIN
- **service** : service de l'utilisateur (ex : "Comptabilité", "RH", "Informatique")
- **date_creation** : date d'ajout (par défaut : aujourd'hui)

Table incident

- **id** : clé primaire, auto-incrémentée

- `titre` : obligatoire
- `description` : obligatoire
- `priorite` : BASSE, MOYENNE, HAUTE ou CRITIQUE
- `statut` : OUVERT, EN_COURS, RESOLU ou FERME
- `date_creation` : date et heure de création
- `utilisateur_id` : clé étrangère vers `utilisateur(id)` (celui qui signale)

Table intervention

- `id` : clé primaire, auto-incrémentée
- `commentaire` : obligatoire (description de l'intervention)
- `duree_minutes` : entier ≥ 0 (temps passé en minutes)
- `date_intervention` : date et heure de l'intervention
- `incident_id` : clé étrangère vers `incident(id)`
- `technicien_id` : clé étrangère vers `utilisateur(id)` (technicien qui intervient)

Fonctionnalités obligatoires

1. Gestion des utilisateurs (CRUD complet)

- Ajouter un nouvel utilisateur (avec choix du rôle)
- Afficher la liste de tous les utilisateurs
- Afficher les détails d'un utilisateur (par ID ou login)
- Modifier les informations d'un utilisateur
- Supprimer un utilisateur (uniquement s'il n'a ni incident ni intervention)
- Rechercher un utilisateur par nom, login ou service

2. Authentification

- Au lancement de l'application, demander login et mot de passe
- Afficher un menu différent selon le rôle (UTILISATEUR, TECHNICIEN, ADMIN)
- Un utilisateur non authentifié ne peut rien faire

3. Fonctionnalités par rôle

UTILISATEUR (celui qui signale les incidents)

- Créer un nouvel incident (titre, description, priorité)
- Consulter la liste de ses propres incidents
- Consulter le détail d'un de ses incidents (voir les interventions)
- Filtrer ses incidents par statut (OUVERT, EN_COURS, RESOLU, FERME)

- Filtrer ses incidents par priorité

TECHNICIEN (celui qui résout les incidents)

- Consulter tous les incidents OUVERTS ou EN_COURS
- Prendre en charge un incident (changer statut à EN_COURS)
- Ajouter une intervention sur un incident (commentaire + durée)
- Résoudre un incident (statut passe à RESOLU)
- Fermer un incident résolu (statut passe à FERME)
- Consulter l'historique des incidents qu'il a traités

ADMIN (superviseur)

- Toutes les fonctionnalités de TECHNICIEN
- Gestion complète des utilisateurs (CRUD)
- Consulter tous les incidents (tous utilisateurs confondus)
- Consulter les statistiques et rapports

4. Workflow des statuts

Les transitions de statut suivantes doivent être respectées :

OUVERT	→	EN_COURS	(prise en charge)
EN_COURS	→	RESOLU	(résolution)
RESOLU	→	FERME	(fermeture)
OUVERT	→	ANNULE	(annulation par utilisateur - optionnel)

5. Rapports et statistiques (pour ADMIN)

- Nombre total d'incidents par statut
- Nombre d'incidents par priorité
- Temps moyen de résolution par incident (en heures)
- Top 3 des techniciens les plus actifs (nombre d'interventions)
- Pour chaque technicien : nombre d'incidents traités et temps moyen par incident
- Taux de résolution dans les 48h (pourcentage d'incidents résolus en moins de 48h)

Architecture technique attendue

Vous devez organiser votre code selon la structure suivante :

`gestion_incidents/`

```

database/
    __init__.py
    config.py          # Configuration BD (mot de passe, hôte, etc.)
    connexion.py       # Singleton pour la connexion

models/
    __init__.py
    utilisateur.py     # Classe Utilisateur
    incident.py        # Classe Incident
    intervention.py    # Classe Intervention

dao/
    __init__.py
    base_dao.py        # Classe abstraite avec méthodes génériques
    utilisateur_dao.py # CRUD Utilisateur + authentification
    incident_dao.py    # CRUD Incident
    intervention_dao.py # CRUD Intervention

menu/
    __init__.py
    auth.py            # Gestion de l'authentification
    interface.py       # Interface utilisateur console

create_tables.py       # Script de création des tables
insert_test_data.py    # Script d'insertion de données de test
main.py               # Point d'entrée principal

```

Contraintes techniques obligatoires

- **Singleton** : La classe de connexion à la base de données doit utiliser le pattern Singleton (une seule instance unique).
- **Héritage** : Une classe abstraite `BaseDAO` doit contenir les méthodes génériques (`get_all`, `get_by_id`, `delete_by_id`) et être héritée par tous les DAO spécifiques.
- **SQL paramétré** : Toutes les requêtes SQL doivent utiliser des requêtes paramétrées (prévention des injections SQL).
- **Gestion des transactions** : Utilisation de `commit()` et `rollback()` pour maintenir l'intégrité des données.
- **Gestion des erreurs** : Toutes les opérations sensibles doivent être encapsulées dans des blocs `try/except`.
- **Respect des conventions PEP 8** : Code indenté proprement, noms de variables explicites, commentaires en français.

Contraintes fonctionnelles

- Un utilisateur ne peut pas être supprimé s'il a des incidents ou des interventions associées.
- Un incident ne peut pas être supprimé s'il a des interventions.
- Seul un technicien ou un admin peut ajouter une intervention.
- Seul l'utilisateur qui a créé l'incident peut voir son détail (sauf technicien/admin).
- Une intervention ne peut être ajoutée que sur un incident dont le statut est OUVERT ou EN_COURS.
- Le statut d'un incident ne peut pas reculer (ex : RESOLU $\rightarrow$ EN_COURS est interdit).

Conseils pour réussir

1. **Commencez par l'authentification** : Avoir un système de connexion fonctionnel vous permettra de tester les différents rôles.
2. **Modélisez le workflow** : Avant de coder, dessinez le diagramme d'états-transitions des incidents.
3. **Itératif** : Développez les fonctionnalités une par une :
 - i) Connexion BD et création des tables
 - ii) CRUD Utilisateur
 - iii) Authentification
 - iv) CRUD Incident (pour utilisateur)
 - v) Prise en charge et interventions (pour technicien)
 - vi) Rapports et statistiques (pour admin)
4. **Conservez l'utilisateur connecté en session** : Utilisez une variable globale ou passez l'objet utilisateur dans les menus.
5. **Testez les permissions** : Vérifiez qu'un utilisateur normal ne peut pas accéder aux fonctions admin.
6. **Versionnez régulièrement votre code sur GitHub** : Faites des commits fréquents avec des messages clairs.

Modalités de rendu et évaluation

Formation des groupes

- Les étudiants forment librement des groupes de 2 ou 3 personnes.
- Chaque groupe désigne un chef de projet responsable du dépôt GitHub.
- Les groupes doivent être constitués au plus tard une semaine après la distribution du sujet.

Rendu 1 : Dépôt GitHub (7 points)

- Créer un repository privé ou public sur `github.com`
- Ajouter l'enseignant comme collaborateur (identifiant : `Muuustafa`)
- Le repository doit contenir :
 - L'intégralité du code source
 - Un fichier `README.md` détaillant le projet
 - Un fichier `requirements.txt`
 - Le script SQL de création des tables
- **Critères d'évaluation GitHub (7 points) :**

Critère	Points
Commits réguliers et significatifs (au moins 10 commits par groupe)	2
Messages de commits clairs et explicites	1
Code complet et fonctionnel	2
README bien structuré (installation, utilisation, captures)	1
Respect de la structure des dossiers	1

Rendu 2 : Présentation orale en classe (14 points)

- Durée : 10 minutes de présentation + 5 minutes de questions
- Tous les membres du groupe doivent participer
- Contenu attendu de la présentation :
 - Présentation du groupe et répartition des tâches
 - Démonstration de l'application (live coding ou vidéo)
 - Explication de l'architecture POO utilisée
 - Explication du workflow des incidents (diagramme d'états)
 - Difficultés rencontrées et solutions apportées
 - Ce que chaque membre a appris

Critères d'évaluation Présentation (14 points)

Critère	Points
Qualité de la démonstration (application fonctionnelle)	4
Explication de l'architecture POO et du code	3
Gestion des erreurs et robustesse de l'application	2
Participation de tous les membres	2
Clarté de l'exposé et supports visuels	2
Réponses aux questions du jury	1

Barème global sur 20 points

Critère	Points
Dépôt GitHub (commits réguliers, structure, README)	7
Présentation orale et démonstration	14
Total	20

Sanctions

- Absence de dépôt GitHub : 0/7
- Absence de présentation (non justifiée) : 0/14
- Retard dans le rendu : pénalité de 2 points par jour
- Plagiat ou copie entre groupes : 0 pour tous les groupes concernés

Attention

Le code sur GitHub doit être versionné régulièrement.
Un dépôt avec un seul commit (le dernier jour) sera fortement pénalisé.

Bon courage et bon travail !